PROYECTO FINAL PROGRAMACION I LABORATORIO

IGNACIO TRAVERSO, NEHUEN DI TULLIO

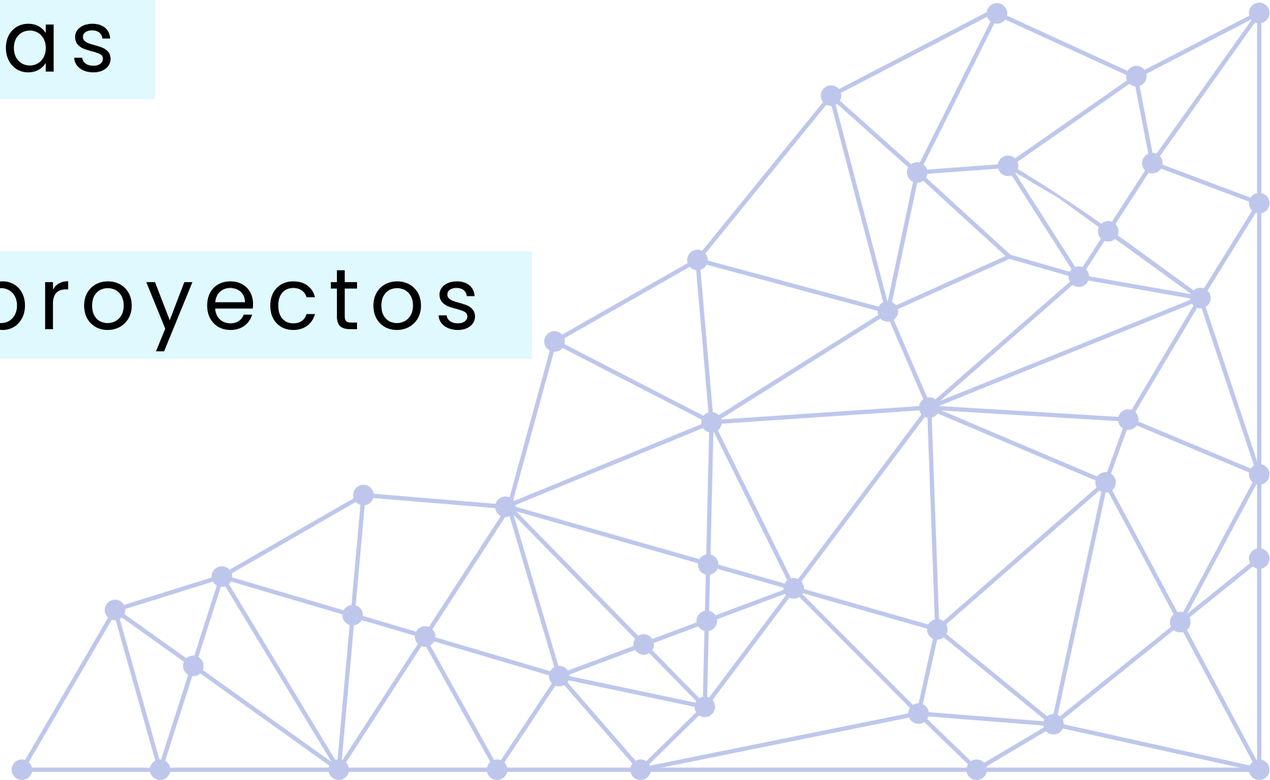
QUE VAMOS A VER ACA?

1. Idea del proyecto
2. Objetivos generales
3. Como nos dividimos las tareas
4. Un poco del desarrollo
5. Funciones Interesantes
6. Pensamientos finales

IDEA DEL PROYECTO

Pensamos en algo:

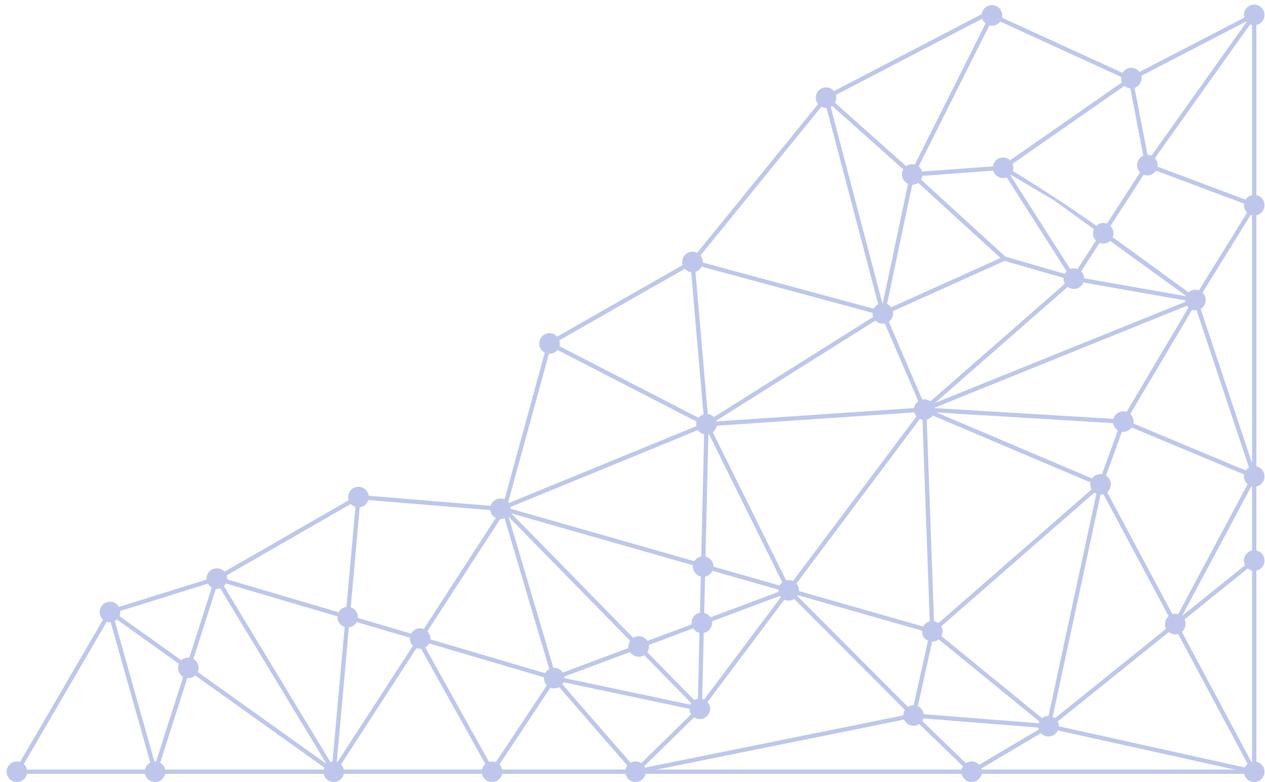
- Simple
- Fácil de dividir tareas
- Divertido
- Inspirado en otros proyectos



DECIDIMOS

Decidimos hacer...

- Un menú de juegos
- 4 juegos diferentes
- 1 juego por persona



PRIMER PROBLEMA

- 2 personas se fueron en el segundo bimestre.
- Ahora tenemos que hacer mucho mas por persona.
- y un alumno solo tiene la materia laboratorio





ACUERDO

- Hablamos con los profes y llegamos a un acuerdo
- Nacho hace su parte en C y Pseint
- Nehuen hace su parte en Pseint
- Achicamos el proyecto a 2 juegos

COMO QUEDO?

Nehuen hace en PseInt el juego de Damas!!!

Nacho hace una idea propia llamada Polyplot!!

El juego de damas internacional funcionando en consola!

Un juego de escribir funciones polinómicas para golpear objetivos en la consola!!

CUANTO NOS LLEVO?

Debido a otras responsabilidades el desarrollo del proyecto tomo su tiempo.

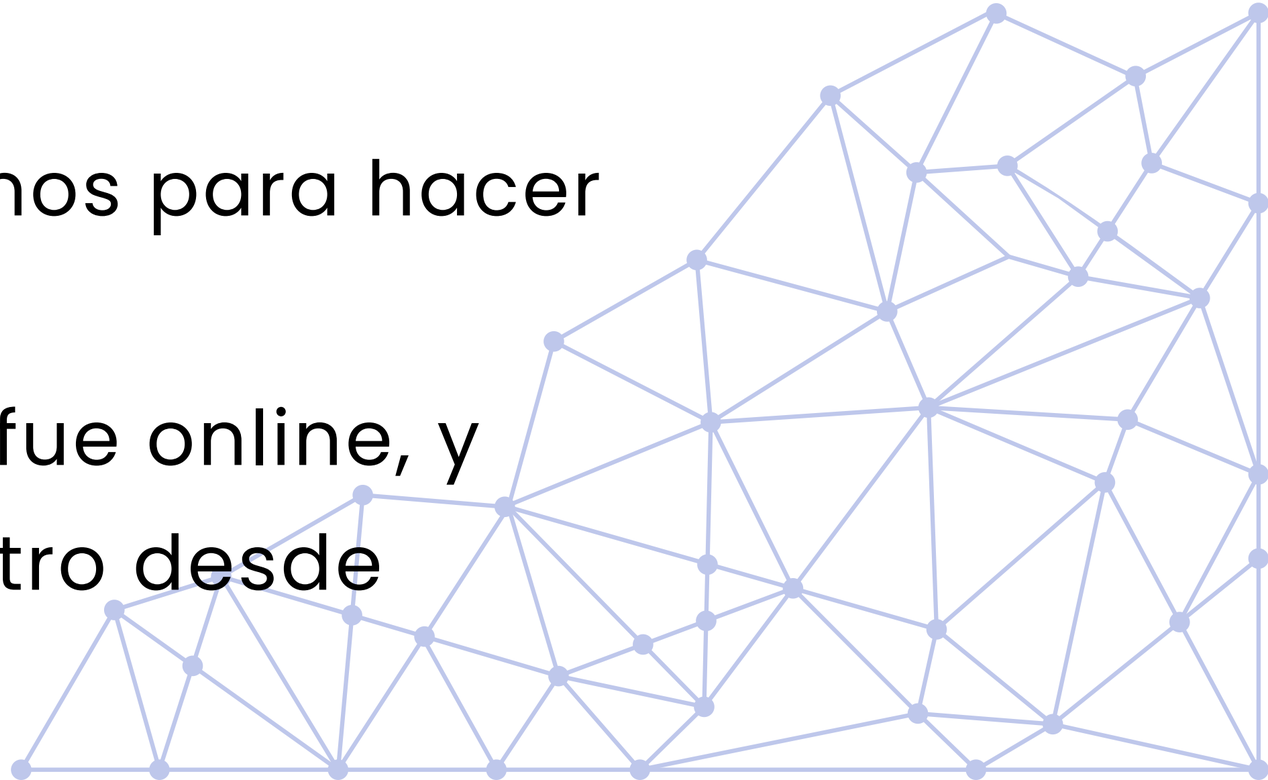
Realisiticamente a cada uno de nosotros nos llevo unas 5 6 sesiones de sentarnos a codificar.



NOS JUNTAMOS...

Nos juntamos para afianzar juntos conceptos de matrices que ambos necesitábamos usar en nuestra parte.

Luego mas al final, coincidimos para hacer la documentación.
El resto de la comunicación fue online, y accedíamos al trabajo del otro desde Github.



FUNCIONES INTERESANTES

POLYPLOT

MostrarGraficoPuntos:

La función mas importante del juego Polyplot:

```
void mostrarGraficoPuntos(punto funcion[], int cantPuntosFuncion, punto objetivos[], int cantObjetivos) {
    char grafico[ALTO_GRAFICO][ANCHO_GRAFICO];
    int f, c;

    //armo el grafico en la matriz
    for (f = 0; f < ALTO_GRAFICO; f++) {
        int y = 25 - f;
        for (c = 0; c < ANCHO_GRAFICO; c++) {
            int x = c - 60;

            if (x == 0 && y == 0) {
                grafico[f][c] = '+';
            } else if (y == 0) {
                grafico[f][c] = '-';
            } else if (x == 0) {
                grafico[f][c] = '|';
            } else {
                grafico[f][c] = '.';
            }
        }
    }

    // dibujo la funcion del usuario en la matriz
    for (int i = 0; i < cantPuntosFuncion; i++) {
        int px = funcion[i].x;
        int py = funcion[i].y;

        if (px >= -60 && px <= 60 && py >= -25 && py <= 25) {
            grafico[25 - py][px + 60] = 'O';
        }
    }
```

```
        // recorro el vector de objetivos y cambio los valores en el grafico correspondientemente, si detecta col
        for (int i = 0; i < cantObjetivos; i++) {
            int ox = objetivos[i].x;
            int oy = objetivos[i].y;

            if (ox >= -60 && ox <= 60 && oy >= -25 && oy <= 25) {
                int col = ox + 60;
                int fila = 25 - oy;

                if (grafico[fila][col] == 'O') {
                    grafico[fila][col] = 'X';
                } else {
                    grafico[fila][col] = '*';
                }
            }
        }

        // imprimir matriz
        printf("\n* = Objetivo  ///  O = Tu Funcion  ///  X = ¡GOLPE!\n");
        printf("+ = Centro    ///  | = (Y=0)       ///  - = (X=0)\n");
        printf(". = Vacio\n\n");

        for (f = 0; f < ALTO_GRAFICO; f++) {
            int y = 25 - f;

            // %3d lo use para alinear los numeros del eje y, eso hace que siempre ocupen 3 espacio, de esa maner
            printf("%3d | ", y);

            for (c = 0; c < ANCHO_GRAFICO; c++) {
                printf("%c", grafico[f][c]);
            }
            printf("\n");
        }
        //Guia de abajo
        printf("      ------------------------------------------------------------------------------------------------------------\n");
        printf("      -60       -50       -40       -30       -20       -10       0         10        20        30        40        50        60\n\n");
    }
```

IngresarFuncionUsuario:

Asegurarnos que el usuario pueda ingresar bien los datos.

```c
void ingresarFuncionUsuario(punto funcion[], int cantPuntos) {
    int grado;
    char confirmacion;

    do {
        int terminos[4] = {0};

        do {
            printf("\nIngrese el grado de la funcion (Maximo 3, Minimo 1): ");

            // Verificamos si fallo leerel entero
            if (leer_int(grado) != 1) {
                while (getchar() != '\n'); // hago esto para evitar que me rompa las pelotas a veces
                grado = -1;
            }

            if (grado > 3 || grado < 1) {
                printf("Entrada invalida. Por favor, ingrese solo un numero entre 1 y 3.\n");
            }
        } while (grado > 3 || grado < 1);

        for (int i = grado; i >= 0; i--) {
            int entradaValida;
            do {
                if (i == grado) {
                    printf("Ingrese el termino principal (x^%d): ",
                } else if (i == 1) {
                    printf("Ingrese el coeficiente lineal (x): ");
                } else if (i == 0) {
                    printf("Ingrese el termino independiente: ");
                } else {
                    printf("Ingrese el coeficiente de x^%d: ", i);
                }
```

```c
            printf("\nSu funcion es: y = ");
            for (int i = grado; i >= 0; i--) {
                if (i == 0) {
                    printf("%d", terminos[i]);
                } else if (i == 1) {
                    printf("%dx ", terminos[i]);
                } else {
                    printf("%dx^%d ", terminos[i], i);
                }

                if (i > 0 && terminos[i - 1] >= 0) {
                    printf("+ ");
                }
            }

            printf("\n¿Confirmar esta funcion? (S/N): ");
            leer_char(confirmacion);

            if (confirmacion == 'S' || confirmacion == 's') {
                for (int i = 0; i < cantPuntos; i++) {
                    int x = i - 60;
                    funcion[i].x = x;
                    funcion[i].y = calcularPuntoFuncion(terminos, grado,
                }
            }

        } while (confirmacion == 'N' || confirmacion == 'n');
```

GenerarPuntosObjetivo:

Solución perezosa y simple a la aleatoriedad de los objetivos

```c
void generarPuntosObjetivo(punto objetivos[], int cantObjetivos) {
    int terminos[4] = {0};
    int grado = (rand() % 3) + 1; // Grado entre 1 y 3 basicamente
    // Coeficientes pequeños (-2 a 2) para evitar que se me vaya fuera de rango la Y
    for (int i = 0; i <= grado; i++) {
        terminos[i] = (rand() % 5) - 2;
    }
    if (terminos[0] == 0 && terminos[1] == 0 && terminos[2] == 0 && terminos[3] == 0) {
        terminos[0] = 1; // Evitar polinomio nulo
    }

    for (int i = 0; i < cantObjetivos; i++) {
        int repetido, xCandidata, yCalculada;
        do {
            repetido = 0;
            xCandidata = (rand() % 121) - 60; // X entre -60 y 60

            // Evitar generar dos objetivos en la misma columna X
            for (int j = 0; j < i; j++) {
                if (objetivos[j].x == xCandidata) {
                    repetido = 1;
                    break;
                }
            }

            if (!repetido) {
                yCalculada = calcularPuntoFuncion(terminos, grado, xCandidata);
                // Si la Y calculada está fuera de los límites, buscamos otra
                if (yCalculada < -25 || yCalculada > 25) {
                    repetido = 1;
                }
            }
        } while (repetido == 1);

        objetivos[i].x = xCandidata;
        objetivos[i].y = yCalculada;
    }
```

FUNCIONES INTERESANTES

DAMAS

cargartablero:

Crear el tablero para usarse en el juego.

```
Funcion cargartablero(table)                // "+" fichas blancas "-" fichas negras "[ ]" espacio vacio
    Definir i, j Como Entero
    Para i<-0 Hasta 9 Con Paso 1 Hacer
        Para j<-0 Hasta 9 Con Paso 1 Hacer
            table[i,j] <- "[ ]"
            Si i mod 2 == 0 y i < 4 y j mod 2 == 1 Entonces
                table[i,j] <- " - "
            SiNo
                Si i mod 2 == 1 y i < 4 y j mod 2 == 0 Entonces
                    table[i,j] <- " - "
                FinSi
            FinSi
            Si i mod 2 == 0 y i > 5 y j mod 2 == 1 Entonces
                table[i,j] <- " + "
            SiNo
                Si i mod 2 == 1 y i > 5 y j mod 2 == 0 Entonces
                    table[i,j] <- " + "
                FinSi
            FinSi
        FinPara
    FinPara
    mostrarTablero(table)
FinFuncion
```

puedeMoverDama:

chequeo constante para verificar los movimientos posibles

```
Funcion valido <- puedeMoverDama(table,fila,columna,fila1,columna1)
    Definir valido Como Logico
    Definir direccionfila, direccioncolumna, filareal, columnareal Como Entero
    valido <- Verdadero

    Si table[fila1,columna1] <> "[ ]" Entonces          //detecta si el destino es una casilla valida
        valido <- Falso
    FinSi

    Si Abs(fila1-fila) <> Abs(columna1-columna) Entonces        //detecta si de verdad esta en diagonal
        valido <- Falso
    FinSi

    Si valido Entonces

        Si fila1 > fila Entonces
            direccionfila <- 1                              //calcula si las filas suben o bajan para las coordenadas
        SiNo
            direccionfila <- -1
        FinSi
        Si columna1 > columna Entonces                      //calcula si las columnas van para la izquierda o la derecha
            direccioncolumna <- 1
        SiNo
            direccioncolumna <- -1
        FinSi

        filareal <- fila + direccionfila
        columnareal <- columna + direccioncolumna               //calcula la posicion de inicio

        Mientras filareal <> fila1 y valido Hacer
            Si table[filareal,columnareal] <> "[ ]" Entonces        //si hay algo que no sea un espacio vacio entremedio del origen y el destino acaba la funcion
                valido <- Falso
            FinSi
            filareal <- filareal + direccionfila
            columnareal <- columnareal + direccioncolumna
        FinMientras
    FinSi
```

puedeComerDama:

chequeo constante para verificar si las damas pueden comer

```
Funcion puede <- puedeComerDama(table,fila,columna,fila1,columna1,turno)
    Definir puede Como Logico
    Definir filadireccion, columnadireccion, realfila, realcolumna Como Entero
    Definir enemigos Como Entero
    puede <- Verdadero
    enemigos <- 0

    Si table[fila1,columna1] <> "[ ]" Entonces
        puede <- Falso
    FinSi

    Si Abs(fila1-fila) <> Abs(columna1-columna) Entonces
        puede <- Falso
    FinSi

    Si puede Entonces

        Si fila1 > fila Entonces
            filadireccion <- 1
        SiNo
            filadireccion <- -1
        FinSi

        Si columna1 > columna Entonces
            columnadireccion <- 1
        SiNo
            columnadireccion <- -1
        FinSi

        realfila <- fila + filadireccion
        realcolumna <- columna + columnadireccion
```

```
        Mientras realfila <> fila1 y puede Hacer

            Si turno = 1 Entonces

                Si table[realfila, realcolumna] = " + " o table[realfila, realcolumna] = " # " Entonces
                    puede <- Falso
                FinSi

                Si table[realfila, realcolumna] = " - " o table[realfila, realcolumna] = " & " Entonces
                    enemigos <- enemigos + 1
                FinSi

            SiNo

                Si table[realfila, realcolumna] = " - " o table[realfila, realcolumna] = " & " Entonces
                    puede <- Falso
                FinSi

                Si table[realfila, realcolumna] = " + " o table[realfila, realcolumna] = " # " Entonces
                    enemigos <- enemigos + 1
                FinSi

            FinSi

            realfila <- realfila + filadireccion
            realcolumna <- realcolumna + columnadireccion

        FinMientras

        Si enemigos <> 1 Entonces
            puede <- Falso
        FinSi

    FinSi

FinFuncion
```

REFLEXIONES:

Fue muy divertido e interesante poder aplicar los conocimientos aprendidos en clase en un proyecto personal.

También pudimos experimentar en primer plano lo conveniente que es usar herramientas como Github para organizar tus archivos y compartirlos.

MUCHAS GRACIAS!!!!